

CareerConnect - Forgot Password AI Prompt Log

Feature implementation, email templates, testing and debugging

Project Request

I am working on CareerConnect, a job recruitment platform developed using SvelteKit, TailwindCSS, Better Auth, Drizzle ORM, PostgreSQL/Supabase, and Resend.

I want to implement a **Forgot Password feature using email verification codes** while maintaining the project's existing structure, coding conventions, authentication system, database schema, and website styling.

Functional Requirements

- 1 Add a "Forgot password?" link to the login page.
- 2 Allow registered users to request a password reset using their email address.
- 3 Send a secure 6-digit verification code through Resend.
- 4 Display: "If an account exists with this email, you will receive a reset code."
- 5 Prevent unregistered email addresses from receiving valid reset codes without revealing whether an account exists.
- 6 Allow users to enter their verification code, new password, and password confirmation.
- 7 Make verification codes expire after 10 minutes, become invalid after use, and allow a maximum of 5 incorrect attempts.
- 8 Enforce a 60-second cooldown between code requests, with each new code invalidating the previous one.
- 9 Display a countdown and disable the resend button until the cooldown expires.
- 10 Preserve the countdown when navigating between pages or refreshing.
- 11 Allow users to return to the reset page without requesting another code.
- 12 Automatically retain the user's email and provide an "Edit email" option.
- 13 Allow users to resend codes directly from the Reset Password page without clearing their password fields.
- 14 Reuse the existing password visibility component and password validation rules.
- 15 Keep the current password unchanged until the reset succeeds.
- 16 Redirect users to login after success and display: "Password reset successfully. Please sign in."

Email Design

Improve both the account verification email and password reset email to match CareerConnect's existing visual identity. Use consistent navy and teal colors, branded headings, responsive layouts, and clear instructions.

The verification email should contain a working verification button, while the password reset email should prominently display the six-digit code and its expiration time. Preserve the existing authentication functionality and Resend integration.

Testing and Debugging

Guide me through implementing the feature step by step, including:

- Identifying and fixing TypeScript, SvelteKit, and authentication errors.
- Investigating navigation, route protection, and redirect problems.
- Verifying environment variables and Resend email delivery.
- Testing registered and unregistered email addresses.

- Testing invalid, expired, previously replaced, and already-used codes.
- Verifying the maximum of five incorrect attempts.
- Checking that resend restrictions cannot be bypassed by refreshing or navigating between pages.
- Confirming password matching, minimum length, and visibility controls.
- Verifying that unsuccessful resets do not change the existing password.
- Confirming successful password changes and login redirects.
- Testing account verification links and confirming verification status in Supabase.
- Investigating and resolving database or authentication errors.
- Creating automated tests for password-reset input validation.
- Running `pnpm check` and `pnpm test` and fixing reported problems.

Implementation Constraints

Reuse existing components and follow the current CareerConnect architecture. Avoid unnecessary dependencies or unrelated changes. Preserve registration, login, logout, email verification, and existing database functionality.

Maintain security through server-side validation, appropriate database permissions, and protection against email enumeration. Explain which files need to be modified, provide the necessary code, and guide me through testing each stage before moving to the next.

Finally, help me verify the complete implementation and prepare the changes for a GitHub commit and pull request.

User Contributions and Additional Requests

During the implementation, I tested the feature step by step to make sure everything worked properly. While testing, I noticed a few issues and asked for fixes and improvements, such as keeping the countdown running when switching pages, disabling the resend button during the cooldown, remembering the user's email, and allowing users to resend a code without losing their entered passwords.

I also wanted the new pages to match the existing CareerConnect design, so I requested the same password visibility feature used on the login page and improved the appearance of both verification and password reset emails.

I tested different situations, including incorrect and expired codes, multiple failed attempts, password changes, and email verification. I also checked the database in Supabase and worked through errors related to email configuration, redirects, and validation.

Finally, I added automated tests and ran `pnpm check` and `pnpm test` to make sure the changes did not affect existing features. **All 27 tests passed, with 0 errors and 0 warnings.**